

Лабораторная работа

«Разработка информационно-поисковой системы. Реализация методов оценки качества ее работы»

Цель работы — освоить на практике основные принципы реализации информационно-поисковых систем и методы оценки качества их работы.

Краткие теоритические сведения.

Информационный поиск — это процесс поиска в большой коллекции (хранящейся, как правило, в памяти компьютеров) некоего неструктурированного материала (обычно — документа), удовлетворяющего информационные потребности.

Поисковый образ документа (ПОД) — описание документа, в виде перечня ключевых слов, которые могут дополняться их весами, связями и указателями роли. По этому описанию внутри системы составляются структуры данных, служащие для поиска документов и выдачи их из хранилищ. Такое же описание строится для пользовательского запроса.

Поисковый образ запроса (ПОЗ) — описание пользовательского запроса, в виде удобном для поисковой системы. Структура поисковых образов для разных поисковых систем может быть различной, однако поисковый образ запроса и поисковый образ документа должны иметь одинаковую структуру в пределах одной поисковой системы.

Основная задача любой поисковой системы – дать пользователю ответ на его запрос или другими словами предоставить список документов релевантных запросу пользователя.

Основными задачами, решаемыми информационно-поисковыми системами, являются:

- приём информации и её предварительная обработка;
- анализ документов и данных, и, возможно, хранение;
- анализ и организация информационных запросов;
- поиск релевантной информации;
- выдача запрашиваемой информации.

Таким образом, поисковые системы обычно состоят из трех компонентов:

- агент (паук, кроулер, робот), который собирает информацию о документах, среди которых будет осуществляться поиск;
- база данных, которая содержит всю информацию, собираемую пауками;
- поисковый механизм, который пользователи используют как интерфейс для взаимодействия с базой данных.

Весовые коэффициенты значимости терминов

Чтобы избавиться от лишних слов и в тоже время поднять рейтинг значимых слов, вводят инверсную частоту термина. Значение этого параметра тем меньше, чем чаще слово встречается в документах базы данных, а вычисляют его по формуле:

$$B_i = \log\left(\frac{N}{P_i}\right), \quad (1.5)$$

где B_i - инверсная частота термина i ;

N - количество документов в базе данных;

P_i - количество документов в базе данных с термином i .

Каждому термину присваивается весовой коэффициент, отражающий его значимость:

$$A_i^j = Q_i^j \times B_i, \quad (1.6)$$

где A_i^j - вес термина i в документе j ;

B_i - инверсная частота термина i ;

Q - частота термина i в документе j .

Современные способы индексирования не ограничиваются анализом перечисленных параметров текста. Поисковая машина может строить весовые коэффициенты с учетом местоположения термина внутри документа, взаимного расположения терминов, частей речи, морфологических особенностей и т.п.

Модель поиска документов

Представление документов и поиск информации в массиве разделим на две модели. Следуя этой логике, векторной будем называть модель описания информационного массива, а линейной - модель поиска информации в массиве. Такое разделение обусловлено тем, что документы записываются в виде двоичных векторов, в то время как поисковые запросы - это линейные преобразования над этими векторами.

В векторной модели информационного потока можно выделить несколько основных понятий: словарь, документ, поток и процедуры поиска и коррекции запросов.

Под словарем понимают упорядоченное множество терминов, мощность которого обозначают как D .

Документ - это двоичный вектор размерности D . Если термин входит в документ, то в соответствующем разряде этого двоичного вектора проставляется 1, в противном же случае - 0. Обычно все операции в линейной модели индексирования и поиска документов выполняются над поисковыми образами документов, но при этом их, как правило, называют просто документами.

Информационный поток или массив L представляют в виде матрицы размерности $N \times D$, где в качестве строк выступают поисковые образы N документов.

$$L = \{ \forall i = \overline{1, N}; j = \overline{1, D} : b_{ij} = \begin{cases} 0, & t_j \notin l_i \\ 1, & t_j \in l_i \end{cases} \}, \quad (1.7)$$

где t_j - термин;

l_i - документ;

b_{ij} - значение в j -й позиции i -го вектора.

При таком рассмотрении можно определить процедуру обращения к информационной системе формулой:

$$L \times q = r, \quad (1.8)$$

где q – вектор запроса;
 r – вектор отклика системы на запрос.

Организация упорядочения и поиска в для разрабатываемой информационно-поисковой системы.

Стратегия логического поиска своей сильной стороной имеет высокую точность, - пользователь получает, только то, что запросил. Сильная сторона данной стратегии является ее же недостатком. Пользователь должен априори знать что в индексе находится, иначе либо его запросы будут приносить слишком мало документов (низкая полнота), либо слишком много (низкая точность). Простейшей стратегией поиска по естественно-языковому запросу является его сведение к логическому, как правило, это стратегия с отказами.

В векторной модели поиска каждому документу d ставится в соответствие вектор

$$D = \{w_{dj}, \dots, w_{dn}\},$$

где w_{dj} - вес j -го ключевого слова в документе d , вычисляемый по формуле нормированного представления TD-IDF:

$$w_{dk} = \frac{N_{dk} \log \frac{N}{N_k}}{\sqrt{\sum_j \left(N_{dj} \log \frac{N}{N_j} \right)^2}}.$$

где N_{dk} — число встреч k -го слова (признака) в документе d ,
 N_k – число документов, содержащих k -ое слово (признак),
 N - общее число рассматриваемых документов.

Аналогично для запроса q вводится одноименный вектор

$$Q = \{w_{qj}, \dots, w_{qn}\},$$

где $w_{qj} = 1$, если j -е слово присутствует в запросе q , и
 $w_{qj} = 0$ в противном случае.

Мера схожести документа d и запроса q в этом случае вычисляется как косинус угла между соответствующими векторами:

$$r(D, Q) = \frac{(D, Q)}{\|D\| \cdot \|Q\|},$$

где (D, Q) - скалярное произведение векторов D и Q ,
 $\|D\|$ и $\|Q\|$ - их евклидовы нормы.

Линейный метод поиска дает возможность отсортировать набор документов по возрастанию релевантностей, что является ключевой задачей при поиске текстов.

Проектирование классов информационно-поисковой системы.

Примерный состав классов для векторной модели поиска информации будет иметь вид, приведенный ниже.

Основной сущностью, над которой происходят действия, является сущность «документ». Следовательно, логично выделить класс Document, с помощью которого можно будет работать с документами. Для того чтобы определить состав класса следует учитывать действия которые применимы к документам. Примерный состав класса Document приведен на рисунке 1.

Document
<pre>+ string title; + string text; + string date; + string time; + int documentID;</pre>
<pre>+ bool AddDocumentToBase(); + static bool DeleteDocumentFromBase(int? documentID); + static bool GetLemmInverseFrequency(int lemmId, out double result); + static bool GetLemmInverseFrequency(string lemmStr, out double result); + static bool GetWordWeightInDocument(string lemmStr, int documentId, out double result); + static bool GetLemmWeightInDocument(int lemmId, int documentId, out double result); + static Dictionary<int, double> GetDocumentVector(int documentId);</pre>

Рисунок 1 — Диаграмма класса Document

Имея вектор документа для реализации поиска необходим еще вектор запроса. Выделим класс Search который будет отвечать за составление поискового образа запроса и непосредственно за составление поисковой выдачи. Диаграмма класса Search изображена на рисунке 2.

Search
<pre>+ bool allWordsTogether + string dateStartString + string dateEndString + string searchQuery;</pre>
<pre>- Dictionary<int, double> GetSearchQueryVector(string query, Dictionary<int, double> docVector) - double ScalarProduct(Dictionary<int, double> a, Dictionary<int, double> b) - double EuclideanNorm(Dictionary<int, double> a) + IEnumerable<SearchResult> GetSearchResult()</pre>

Рисунок 2 — Диаграмма класса Search

SearchResult – класс представляющий результат поиска. Один объект класса содержит информацию об одной ссылке на документ в поисковой выдаче. Структура класса приведена на рисунке 3.

SearchResult
<pre>+ int documentId; + string title; + string snippet; + double rank; + string date;</pre>

Рисунок 3 — Диаграмма класса SearchResult

Рассмотрим каждое поле класса SearchResult

- *int documentId* – идентификатор документа из таблицы Document.
- *string title* – заголовок документа.
- *string snippet* – фрагмент текста документа (первые 300 символов).
- *double rank* – релевантность документа запросу.
- *string date* – дата добавления документа в базу.

Требования к разрабатываемой системе

- ✓ на входе – множество естественно-языковых текстов по которым осуществляется поиск;
- ✓ выделение ключевых слов документов осуществляется системой автоматически в соответствии с формулой 1.6;
- ✓ система должна позволять пользователю формулировать ЕЯ-запрос;
- ✓ на выходе – список документов, релевантных запросу пользователя, в соответствии с моделью поиска, согласно варианту;
- ✓ результаты поиска должны содержать: активную ссылку на документ, список слов запроса присутствующих в документе.
- ✓ на основании информации о существующих метриках, наиболее часто используемых для оценки качества работы систем информационного поиска (см. 2004_gomir_metrix.pdf), требуется дать оценку работы СИП. Вычисление оценок, получаемых на основании метрик, реализовать программно, путем вызова соответствующего подменю, и отображать в виде таблиц и графиков
- ✓ интерфейс системы должен быть предельно простым и доступным для пользователей любого уровня, содержать понятный набор инструментов и средств, а также help-средства.

№ варианта	Реализация элементов ИИ	Сфера применения	Стратегия поиска	Язык
1	Интерфейс с пользователем	Локальная вычислительная сеть	Логическая	Русский
2	Интерфейс с пользователем	Сеть Интернет	Логическая	Русский
3	Интерфейс с пользователем	Локальная вычислительная сеть	Векторная	Русский
4	Интерфейс с пользователем	Сеть Интернет	Векторная	Русский
5	Интерфейс с пользователем	Локальная вычислительная сеть	Вероятностная	Русский
6	Интерфейс с пользователем	Сеть Интернет	Вероятностная	Русский
7	Интерфейс с пользователем	Локальная вычислительная сеть	Логическая	Английский
8	Интерфейс с пользователем	Сеть Интернет	Логическая	Английский
9	Интерфейс с пользователем	Локальная вычислительная сеть	Векторная	Английский
10	Интерфейс с пользователем	Сеть Интернет	Векторная	Английский
11	Интерфейс с пользователем	Локальная вычислительная сеть	Вероятностная	Английский
12	Интерфейс с пользователем	Сеть Интернет	Вероятностная	Английский
13	Модуль индексирования документов	Локальная вычислительная сеть	Логическая	Русский

14	Модуль индексирования документов	Сеть Интернет	Логическая	Русский
15	Модуль индексирования документов	Локальная вычислительная сеть	Векторная	Русский
16	Модуль индексирования документов	Сеть Интернет	Векторная	Русский
17	Модуль индексирования документов	Локальная вычислительная сеть	Вероятностная	Русский
18	Модуль индексирования документов	Сеть Интернет	Вероятностная	Русский
19	Модуль индексирования документов	Локальная вычислительная сеть	Логическая	Английский
20	Модуль индексирования документов	Сеть Интернет	Логическая	Английский
21	Модуль индексирования документов	Локальная вычислительная сеть	Векторная	Английский
22	Модуль индексирования документов	Сеть Интернет	Векторная	Английский
23	Модуль индексирования документов	Локальная вычислительная сеть	Вероятностная	Английский
24	Модуль индексирования документов	Сеть Интернет	Вероятностная	Английский
25	Модуль отбора документов	Локальная вычислительная сеть	Логическая	Русский
26	Модуль отбора документов	Сеть Интернет	Логическая	Русский
27	Модуль отбора документов	Локальная вычислительная сеть	Векторная	Русский
28	Модуль отбора документов	Сеть Интернет	Векторная	Русский
29	Модуль отбора документов	Локальная вычислительная сеть	Вероятностная	Русский
30	Модуль отбора документов	Сеть Интернет	Вероятностная	Русский
31	Модуль отбора документов	Локальная вычислительная сеть	Логическая	Английский
32	Модуль отбора документов	Сеть Интернет	Логическая	Английский
33	Модуль отбора документов	Локальная вычислительная сеть	Векторная	Английский
34	Модуль отбора документов	Сеть Интернет	Векторная	Английский
35	Модуль отбора документов	Локальная вычислительная сеть	Вероятностная	Английский
36	Модуль отбора документов	Сеть Интернет	Вероятностная	Английский

В отчет по работе необходимо включить:

- ✓ структуру разработанной системы;
- ✓ структуру базы данных системы;
- ✓ основные алгоритмы реализации компонентов системы;
- ✓ результаты тестирования системы;
- ✓ информацию о тестовой коллекции документов;
- ✓ результаты оценки по каждой из метрик (аналитически и графически);
- ✓ результаты анализа полученных данных, и предложения по улучшению работы СИП;

- ✓ описание и особенности применения готовых к использованию компонент (библиотек, классов, фреймворков и т.п.) в случае их использования в работе.